Universidad Nacional de Loja

Facultad de la energía, las industrias y los recursos naturales no renovables

Sistemas Digitales

AA 01 – Unidad 01

AA 1 – U1



Docente:

- Mgtr. Sinche Cueva Luis Dario

Estudiante:

- Buri Camacho María Soledad

Fecha:

- 11 de noviembre, 2026

Loja - Ecuador

Sistemas Digitales

Actividad Autónoma.

30 / sep / 2026

Sistemas embebidos: una sola trabajo ejecuta.

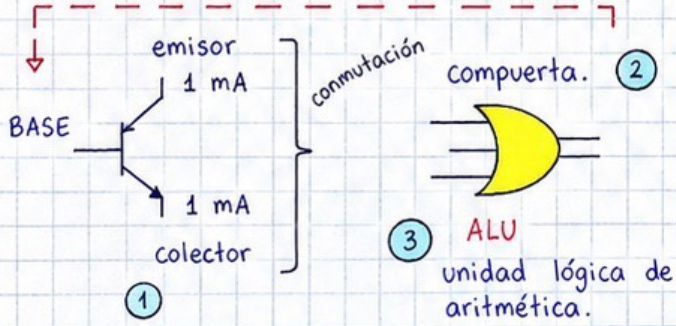
Microcontrolador: (corazón de estos sistemas) una función tiene:
- CPU
- memoria
- periféricos

Microcontrolador / CPU / memoria / periféricos)
- 8-32 bits (max)
- o menos (Flash + SRAM)
- control dedicado
- embebido
- 400 MHz

1.1	5V	5V 3.3V
0.1	0V	0V

Microprocesador: CPU
- 64 bits (max)
- Externa (RAM, HDD / SSD)
- Propósito general
- PC (servidores)

me entrega



Preguntas

1. ¿Qué es la memoria?

Componente que permite almacenar información en forma de bits.

estados 0 y 1

Información almacenada:
- datos
- instrucciones de un programa
- direcciones
- resultados temporales,
- configuraciones del sistema.

La memoria contiene:

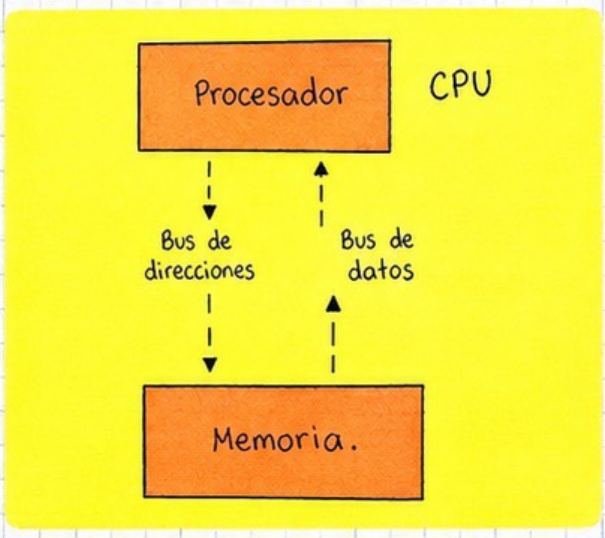
① dirección — donde está almacenado el dato.

② dato — qué información está almacenada.

- **Bus de datos**

Conjunto de líneas de comunicación física (cables) que permite transportar los datos entre la memoria y otros componentes, principalmente el procesador.

- El ancho del bus de datos determina cuántos bits pueden transferirse simultáneamente: 8 / 16 / 32 / 64 → 8 bits



CPU:
- Bus de direcciones: ¿dónde?
- Bus de datos: ¿qué dato?
- Bus de control: ¿Qué operación?

3. **Registros:** pequeña cantidad.

También almacenan bits pero están dentro del procesador y tienen una función mucho más inmediata.

tienen en común: registros ---→ guarda datos

4 tipos de memoria

Memoria volátil

- Pierde la información cuando se corta la alimentación eléctrica.

ejemplos:
- RAM
- SRAM
- DRAM

Memoria no volátil

- Conserva la información incluso cuando se apaga el sistema.

ejemplos:
- ROM
- PROM
- EPROM
- EEPROM

} memoria flash

RAM → random Access Memory

Memoria que permite acceder a cualquier posición de memoria utilizando su dirección.

encontramos:

1. SRAM → static RAM { circuitos biestables para almacenar los bits.

Flash

características:
- Muy rápida
- Más cara
- Consume más área por bit
- Se utiliza en memoria caché

2. DRAM → dynamic RAM — Utiliza celdas basadas en capacitores.

Características:
- Mayor densidad
- Más económica por bit
- Más lenta que SRAM
- Refresco periódicamente

6. ROM → read only memory.

Memoria de solo lectura, la idea fundamental es que la información almacenada permanece aunque se retire la alimentación.

ROM:
- PROM
- EPROM
- EEPROM
- Flash

} familia

7. ¿Qué es EPROM?

memoria de solo lectura programable y borrable.

Es una memoria no volátil, es decir mantiene la información sin alimentación eléctrica.

EPROM puede ser:
- Ser programada.
- Mantener la información por borrado.
- Volver a ser programada.

---

7 / Oct / 2026

SO → Linux
Tarea → Páginas web
Binario → Todo.

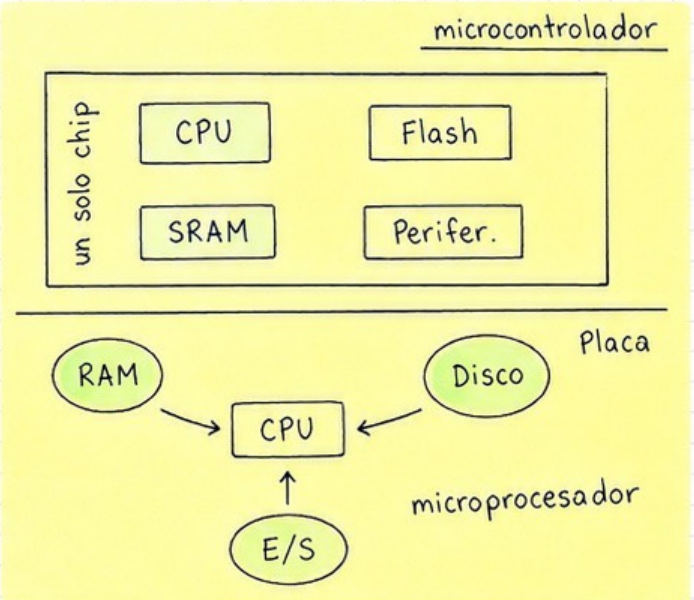
Se guardan los datos en la memoria mediante Flip-Flop / capacitor, utilizando una estructura donde almacena - mantiene - almacena - cancela.

Ensamblador

- Lenguaje de bajo nivel.

¿Qué es un sistema embebido?

- Aparato con un microcontrolador dentro dedicado a una tarea fija y con recursos limitados.



00 → suma
and → multiplica
se conecta

microcontrolador

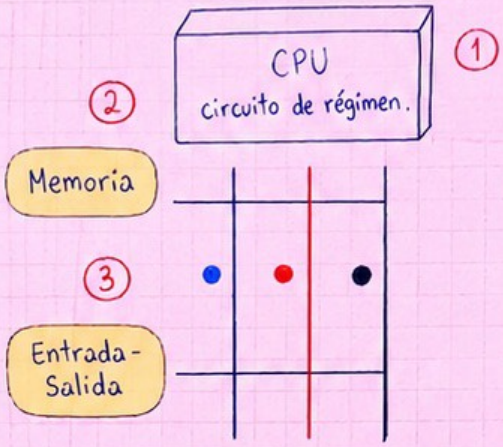
- 3 pasos unidos por buses
- CPU → ejecuta el programa

tipos
Bits 5150
serial ingrese serial chip
5109
P200
9700
paralelo entro paralelo solo

linea de conexion



(2) E/S
(3) memoria



dentro de la CPU

ALU lógica aritmética / operaciones materm.
Unidad de control: lee y decide que hace
Registros: pequeños internos almacena bits
Contador de programa: guarda la dirección de la siguiente instrucción
núcleo = una CPU completa.

CPU repite tres pasos al ritmo del reloj

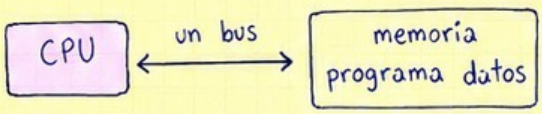
Busca la línea → Buscar → Trae la instrucción de memoria (Flash)
decodifica la línea → Decodificar → Decodificar → entiende que hacer.
ejecuta la línea → Ejecutar → 16 MHz → la velocidad del pulso para registrar cada instrucción. 62,5 ns

Ejemplo leer y escribir un byte

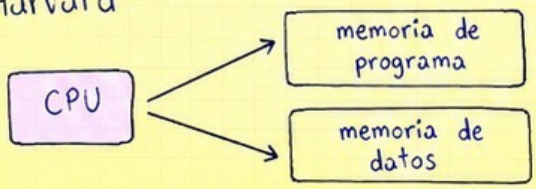
	Escribir 0010 0000 en 0×25	Leer la dirección 0×25
1	La CPU pone 0×25 en el bus de dirección	La CPU pone 0×25 en el bus de direcciones
2	Pone 0010 0000 el bus de datos	Activa leer en el bus control
3	Activa "escribir" en el bus de control	La memoria pone el contenido en el bus de datos.
4	La memoria guarda el byte.	La CPU captura el byte.

arquitecturas

von Neumann



Harvard



↳ aunque la siguiente instrucción llega mientras se usa un dato.

Memorias

Flash → Guarda el programa, se reescribe al subir un programa el IDE como un libro.

SRAM → Guarda las variables mientras el programa corre. Rápida y pequeña.
como una pizarra.

EEPROM → Guarda pocos datos que el programa cambia y deben sobrevivir al apagado.
como una libreta.

ESP32 → inicia el trabajo dentro de otro pequeño.

Ciclo: borrar y volver a escribir la misma celda.

128	64	32	16	8	4	2	1

Periféricos

(1) GPIO

nivel	escribe	arduino uno	ESP32
Bajo	0	≈ 0V	≈ 0V
Alto	1	≈ 5V	≈ 3,3V

* Tener cuidado con la zona de incertidumbre.

Diferencias Atmega 328P

	ATmega 328P	ESP32
Fabricante	Microchip	Espressif
Placa típica	Arduino Uno	ESP32 DevKit V1
Bits y núcleos	8 bits, 1 núcleo (solo tarea)	32 bits, 2 núcleos (2 en paralelo)
Reloj	16 MHz en el Uno	Hasta 240 MHz
Voltaje de los pines	5 V	3,3 V (no tolera 5 V)
Wi-Fi y Bluetooth	NO	Sí
		1 sistema operativo único

Chip, módulo y placa

Placa: ESP32 DevKit V1
uso, regulador, botones en "boot", etc.

Módulo ESP32-WROOM-32
chip, flash y antena.

Chip ESP32

AND : 1 si solo si ambos son 1
OR : 1 si alguno es 1
XOR : 1 si son distintos
NOT : invierte.

Las 4 recetas

Poner el bit n en 1

```
REG |= (1 << n);
```

OR con 1 da 1, OR con 0 deja igual.

Poner el bit n en 0

```
REG &= ~(1 << n);
```

~ invierte la máscara, AND 0 da 0.

Invertir el bit n

```
REG ^= (1 << n);
```

XOR con 1 invierte, XOR con 0 deja.

Consultar por el bit n

```
if (REG & (1 << n));
```

Queda distinto de 0 solo si el bit es 1.

Convertir



Semana 1
Sesión 2

1 0 0 0 0 0 0 1
8 4 2 1 8 4 2 1

0x81 decimal: 129

200

1 1 0 0 1 0 0 0

0xC8

10 A
11 B
12 C

registros periférico DDRB PORTB PINB
registros memoria (primero)

RISC / CISC } set de instrucciones
- microcontrolador
- protocolo
- ensamblador

juego de instrucciones del AVR

LDI = carga inmediatamente registros
ADD = suma registros
↳ registro en el primer registro.

SBI = escribe un 1
CBI = escribe un 0
RJMP = salto de una línea a otra.

Cómo funciona

1. Lenguaje C

```
PORTB |= (1<<5)
```

2. Ensamblador

```
sbi  0x05, 5
```

3. Código máquina en la flash

```
1 001 010 000 101
```

(0x9A20)

Tres registros por puerto

DDRB Dirección: entrada o salida / salida.

0	1
entrada	salida
escuchando	envía

escucha o habla.

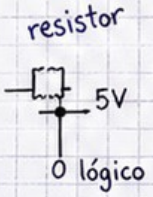
PORTB Salida: el valor que sale.
Entrada: activa la pull-up.

0	1
0 voltios	5 voltios

si en el pin la pull-up activa.

Ejemplo:

DDRB = 0
PORTB = 0
PORTB = 1
} activa resistencia pull-up. siempre recibe un 1 lógico.



DDRB = 0
PORTB = 0 → conectar sensores analógicos.

A, Z alta impedancia.

PINB lee el voltaje real del pin.

0	0	entrada flotante
0	1	entrada con pull-up
1	0	salida 0 (0V)
1	1	salida en 1 5V

ODR PORT

Ejemplos

① 0x81 a binario

1	0	0	0
8	4	2	1

0	0	0	1

1 0 0 0 0 0 0 1

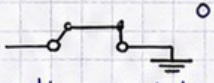
② Escribe la línea que apaga el bit 4 de PORTB.

```
PORTB &= ~(1<<4)
```

③ Con DDRD2 = 0
PORTD2 = 1

¿Qué lee PIND2 con el botón pulsando GND?

DDRD2 = 0
PORTD2 = 1
} activa resistencia pull-up.



Lee un 0 lógico.

Semana 02
sesión 01

Atmega 328P

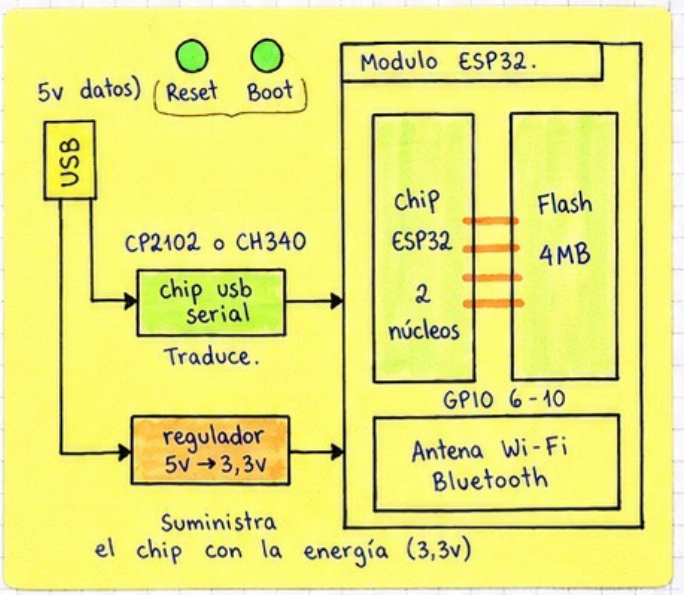
- 8 bits
- 1 núcleo 16 MHz
- 2KB SRAM
- 32K dentro del chip FLASH
- 5 V
- Fijo (PB5 = bit 5 de PORTB) en periférico.

ESP32 - WROOM - 32

- 32 bits
- 2 núcleos + tensa
- 520 KB SRAM
- 4 MB chip aparte modulo (SPI)
- 3,3 V
- configurable (GPIO matrix)

Un (pin) se controla escribiendo bits en el registro asociado y lee datos.

registros mapeados en memoria.



pin ≠ periférico

- conjunto interno que cumple una función específica.
- Puerto físico que conecta el mundo físico con el exterior de un microcontrolador.

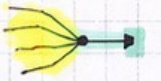
ESP32: Central configurable.

Multiplexor de entradas y salidas.

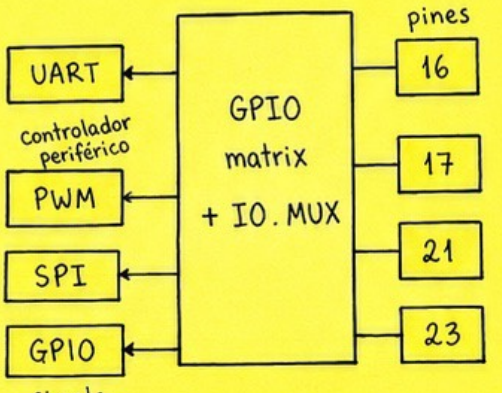


GPIO

Entrada I
Salida O
Propósito P
General G



bit 5 de PORTB → PB5



in → leer.

WA1S → set

Bits en 1 encienden.
Bits en 0 no se toca.

WATC → limpiar.

Bits en 1 apagan.
Bits en 0 no se toca.

GPIO_OUT	0	1	0	1
escribo en WATS	0	0	1	0
GPIO_OUT	0	1	1	1

Una sola escritura sin leer antes la operación es atómica.

Configuración en pin.

1. Elegir su función (IO_MUX)
2. Conectar la señal a la matrix.
3. Habilitar la salida.



Sketch → convierte, planifica o escribe código.



Capas de abstracción

Tú código	lo que tú escribes (programación)	más fácil ↓
Cód. de alto nivel	lenguaje digitalWrite, read, C++, etc.	
ESP-IOF	gestión de E/S (pines)	
Registro	control de bits, configura funciones	
Hardware	Circuitos internos, transistores, memoria, etc.	más control y velocidad

MAPA DE PINES

GPIO	Función/uso
2,4,5	Pulsadores

- Seguros para leds y pulsadores.
- el ESP32 los lee cuando arranca pero saber en qué modo arranca
- si los conectas mal con un led, se hace GND al arrancar y puede no verlo.
- Solo entrada, sin pull up (solo botones) Solo leer.
- Si se quieren usar sin afectar la salida, hay que configurar el pin.
- UART0, no debe se carga el código y debe estar libre (print).
- Alimentación / no son programables.

Límite eléctricos

$R = \frac{3{,}3\ V}{I}$ — A → corriente que pasa

3,3 V → lo que da

I → lo que consume el led

- 10 mA por pin
- 3,3 V

Del reset / loop()

Vin	entrada de voltaje
3V3	salida de voltaje regulado

Bootloader

Bootloader ROM	• Se leen las partes del chip (pines).
Bootloader flash	• Si el boot dice "todo bien", arranca carga el S.O. (sistema) del chip en la flash (memoria).
arranque ESP-IOF	• Inicializa el programa, revisa la memoria disponible, los relojes y prepara el sistema.
Librerías (Sketch / Núcleo)	• Código base del chip (funciones listas: leer, escribir, etc.)
Setup() (una vez)	• Se ejecuta solo una vez antes de empezar.
Loop() (siempre) ↩	• Que contiene en bucle infinito dentro de las tareas.

Tareas principales:
1. Iniciar.
2. Ejecutar.
3. Repetir (sin fin).

variables locales:

ESP32 → trabaja en 32 bits.

Semana 2 sesión 2



Resistencia = carga

3 elementos, cuando interactúan, se deben ajustar.

Una variable es una caja

uint8_t → sin signo / positiva.

int8_t → con signo / negativa.

Tipos stdint.h

- registros y máscaras — uint8_t
- protocolos y tramas — uint8_t / uint16_t
- tiempos — uint32_t

U = sin signo

número = bits

_t = tipo

puede = frecuencia, de los dos el mismo tiempo (temporización)

ver los tipos

stringing → ver el estado actual.

$$T = \frac{1}{f} \qquad f = \frac{1}{Ts}$$

Desbordamiento

Es el alcance.

tope → 255 → 0
+1

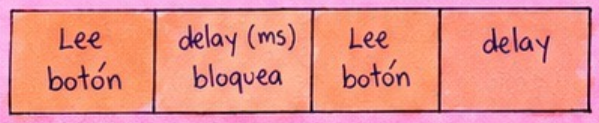
Volátil: Puede cambiar sin que lo leas.

Static: conserva su valor.

---

delay

loop () con delay (1500)
↓
no hago nada
hasta que pase este delay.



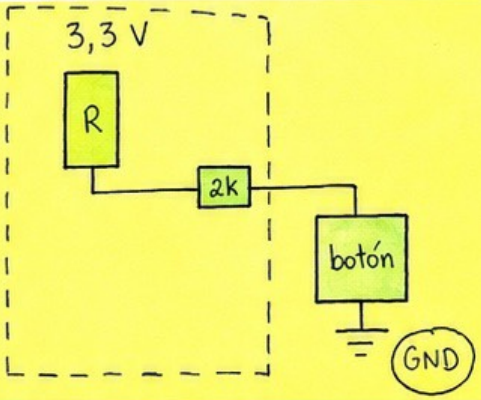
↓

presiona → nadie lo leyó.

millis () toma tiempo de reloj
toma tiempo de ejecución.

- hago otras acciones, pero puedo saber el tiempo sin que se detenga el resto.

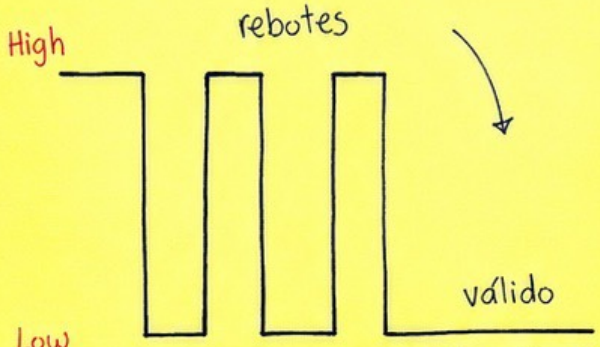
Pull - UP



Pulsador	Suelto HIGH (1)
	presionado Low (0)

el rebote

Acepta el cambio solo si la señal se queda quieta entre 20 y 50 ms.



Temporizadores de hardware

- Circuito dentro del microcontrolador que cuenta pulsos de reloj, periférico hardware.

delay: Esperan sin hacer nada.

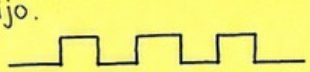
millis: Registra constantemente.

Temporizador: Un circuito cuenta y avisa la tarea exacta.
- preciso de almacenamiento.

Contador: Es un registro del temporalizador con un circuito sumador pegado.
le suma 1 en cada pulso.

CPU ↳ vía bus { Donde inicia, cuando inició, cuando terminó } contador

Reloj: señal que sube y baja a un ritmo fijo.



Avanza:
- Periférico:
- Bus

Del como se comunica todo el circuito.
CPU - Periféricos

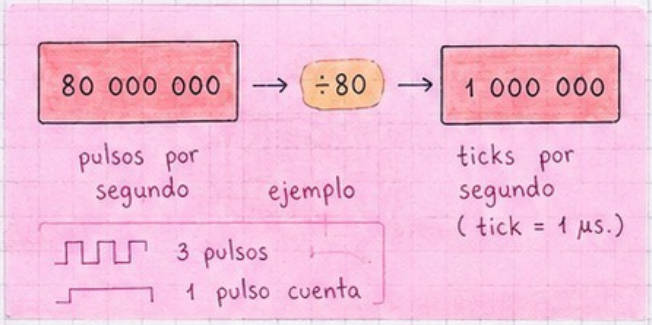
80 MHz
80 millones de veces en 1 segundo.

Semana 3



U.10 11-01 aldo.

Prescale: Divisor de reloj pasaría al contador solo 1 de cada N pulsos de reloj.



Prescale ticks ↪ 1 pulso

pequeño
+ preciso
• el contador se llena más pronto

grande
- preciso
- más memoria

frecuencia (cantidad de pulsos)

Resolución: Duración de un tick. Es el tiempo más pequeño que el temporizador puede distinguir.

resolución = período

$$\left[\frac{1}{f} \right]$$



altura de un peldaño = resolución

número de peldaños = alarma

Alarma: número que se guarda en un registro

alarma = tiempo x f ④

contador – alarma

registro registro

Cuando el contador alcanza la alarma el temporizador genera un aviso.

ejemplo 1.

Datos: f = 1 000 000 ticks por segundo
tiempo deseado = 0,5 s.

① **Resolución**

$$\frac{1}{f} = \frac{1}{1\,000\,000} = 1\ \mu s$$

② **Alarma**

k = tiempo · f = 0,5 s x 1 000 000 =

500 000 ticks

③ **Comprobación**

500 000 x 1 µs = 0,5 s.

④ **Prescaler**

$$N = \frac{80\,000\,000}{1\,000\,000} = 80$$

ejemplo 2.

ejemplo base

f = 10 000 ticks por segundo
alarma suene = 2 500 ticks

① **Resolución**

R = 1 / f

$$R = \frac{1}{10\,000} = 0{,}0001\ s = 1 \times 10^{-4}\ s$$

0,1 milisegundos
100 µs.

② **Tiempo**

t = alarma x resolución
t = 2 500 x 0,0001
t = 0,25 s.

③ **Prescaler**

$$N = \frac{80\,000\,000}{10\,000} = 8\,000.$$

el contador va llegó a la alarma

Autorecarga: el aviso se repite solo al alcanzar la alarma, el contador vuelve a 0 ↓

Desbordamiento: el máximo que cabe en los bits

- el contador "da la vuelta" a 0
- sin mirar la alarma, puede no cumplir regreso.

☑ SFP32 y temporizador.

TIMG0

Timer 0	Timer 1
contador 64 bits	contador 64 bits
prescaler 16 bits	prescaler 16 bits

TIMG1

Timer 0	Timer 1
contador 64 bits	contador 64 bits
prescaler 16 bits	prescaler 16 bits

Interrupción: señal que un periférico envía a la CPU "delante".

ISR: Rutina de servicio de interrupción.

Interno: El temporizador llega a la alarma.

Callback: función que yo escribo pero no la llamo.

- Con lo que quiero que pase.

Semana 4



Sondeo: Preguntar una y otra vez si algo ya pasó.

Interrupción: Aviso del hardware que pausa el programa (tiempo, alarma, /periférico).

Interrupción externa: Interrupción que llega desde fuera del chip (con un botón) /no definido.

ISR: Función que se ejecuta al llegar el aviso.

Flanco: subida y bajada señal cambia 0 ↑ 1, 1 ↓ 0.

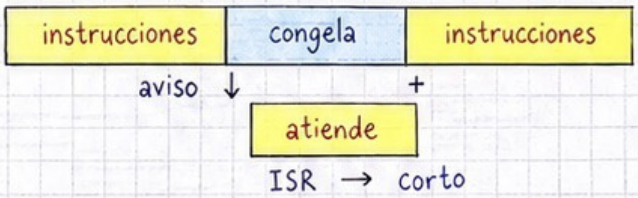
Rebote: Varios cambios falsos.

Watchdog: < ISR → Instrucciones / Task

Reinicia el chip si algo se queda trabado.

Interrupción: Como instalar un timbre. Avisa.

Loop:



1. Botón = presiona.
2. Pin GPIO 21 = cambia 1 a 0.
3. Periférico GPIO = detecta el cambio de hardware.
4. CPU = pausa el loop y corre la ISR.

Nivel de flanco

Nivel: Tiempo que tarda en cambiar un estado.

Flanco: Este instante.



PULL-UP = siempre da un 1

tres modos

cambio

Falling 1 → 0
Flanco de bajada, presiona.

Rising 0 → 1
Ocurre al soltar.

Change
Los dos estados toma en cuenta la ISR, se ejecuta dos veces.

Setup ()

El timbre se instala una vez.

```
attachInterrupt ( digitalPinInterrupt(21),
                  ISR, modo);
```

primero pinmode, después attachInterrupt.

ISR Función que nunca llamas, la llama el hardware cuando llega el aviso (interrupción).

↓

Rutina de servicio interrupción

Reglas:

- Subir una bandera.
- Guardar el tiempo millis.
- Sumar 1 contador.
- digitalWrite (es rápido).

} Rápida.

Enciende el led en el pin 21 o bien.

IRAH Instrucciones RAH
Código que la CPU ejecuta.

ORAH Guarda datos.

Función en la IRAH

```
void IRAH_ATTR ISRButton();
  // código corto, ISR @ IRAH_ATTR
                    siempre.
```

Función que estará en IRAH.

Variable: bool alerta = false.

No confíes en la copia: se usa la RAH cada vez que la RAM la use.

Flash: → Memoria en la que el programa se guarda.
Se encarga de subir un programa.

SRAM: → Guarda variables y datos, el programa corre se borra.

Caché: → CPU guarda copias temporales de datos para acceder más rápido.

ISR Button () → función que dice vamos a interrupciones.